

MoMo SMS Transactions

REST API — Building & Securing (Week 3)

Author: Endahiro (solo contributor)

Runtime: Python 3.10+ standard library (http.server, no third-party framework)

Dataset: modified_sms_v2.xml, 1,691 SMS records → 1,683 transactions after filtering OTPs

Contents: API security introduction, endpoint documentation, DSA comparison results, reflection on Basic Auth limitations.

This project is being completed individually with the instructor's approval.

1. Introduction to API Security

A REST API is the door through which a client — a mobile app, a web dashboard, or a third-party integration — reaches the data it needs. That makes the API the primary attack surface of any modern system. Securing it well means answering three related questions on every request: who is asking (authentication), what are they allowed to do (authorization), and how do we know the message hasn't been tampered with in transit (integrity).

This assignment focuses on the first question, authentication, using the simplest scheme HTTP defines: **Basic Authentication**. Basic Auth is a good pedagogical starting point because every language and every HTTP client supports it out of the box, and its weaknesses are easy to see once you've built it. The API implemented for this project accepts a username and password on every request, checks them against a configured pair, and returns 401 Unauthorized when they don't match — exactly what the assignment requires. Section 4 discusses why this scheme is not suitable for production and what a real deployment would use instead.

The remaining security considerations — authorization tiers (admin vs read-only), integrity (HTTPS/TLS), rate limiting, input validation, and audit logging — are all layered on top of authentication. The API here validates JSON bodies and rejects malformed input with 400 Bad Request, and every request is logged to stderr for basic auditability.

2. Endpoint Documentation

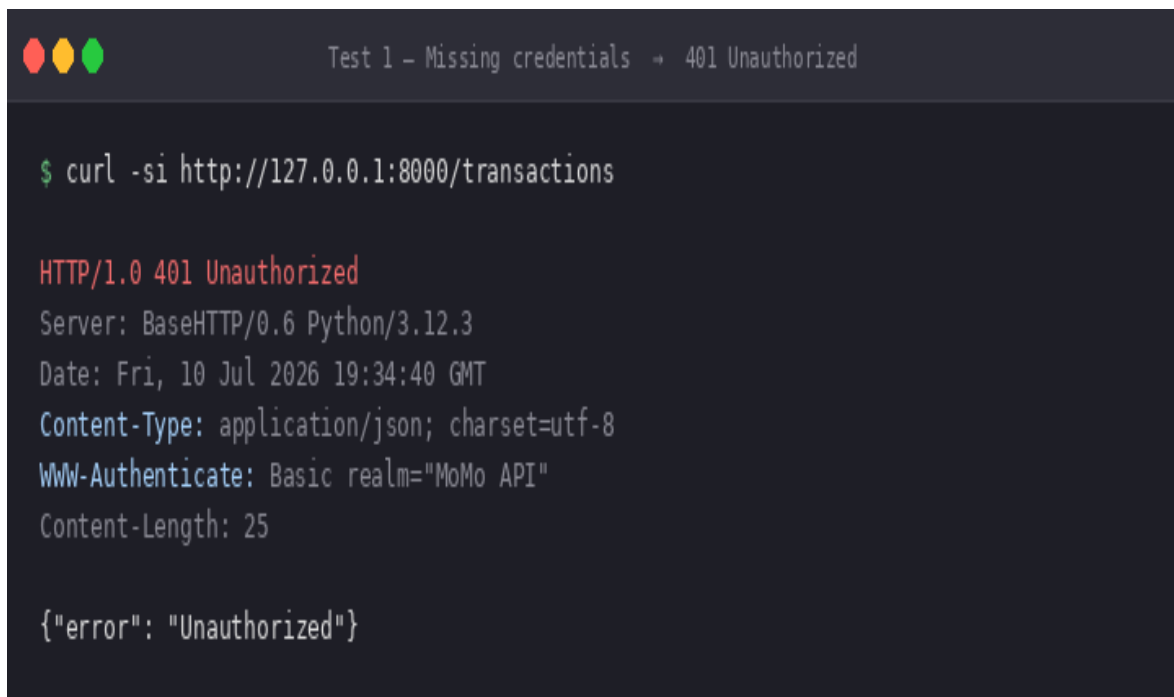
The API exposes five CRUD endpoints under `/transactions`. All require Basic Auth. Full request/response examples with error codes live in `docs/api_docs.md`; this section is a reference summary.

Method	Path	Purpose	Success	Errors
GET	<code>/transactions</code>	List all transactions	200 OK	401
GET	<code>/transactions/{id}</code>	Fetch one transaction	200 OK	401, 404
POST	<code>/transactions</code>	Create a new transaction	201 Created	400, 401
PUT	<code>/transactions/{id}</code>	Update a transaction	200 OK	400, 401, 404
DELETE	<code>/transactions/{id}</code>	Remove a transaction	200 OK	401, 404

2.1 Live testing evidence

Every endpoint below was exercised against the running server. All screenshots were captured directly from a Bash session using curl.

Missing credentials — 401 Unauthorized



```
Test 1 - Missing credentials → 401 Unauthorized

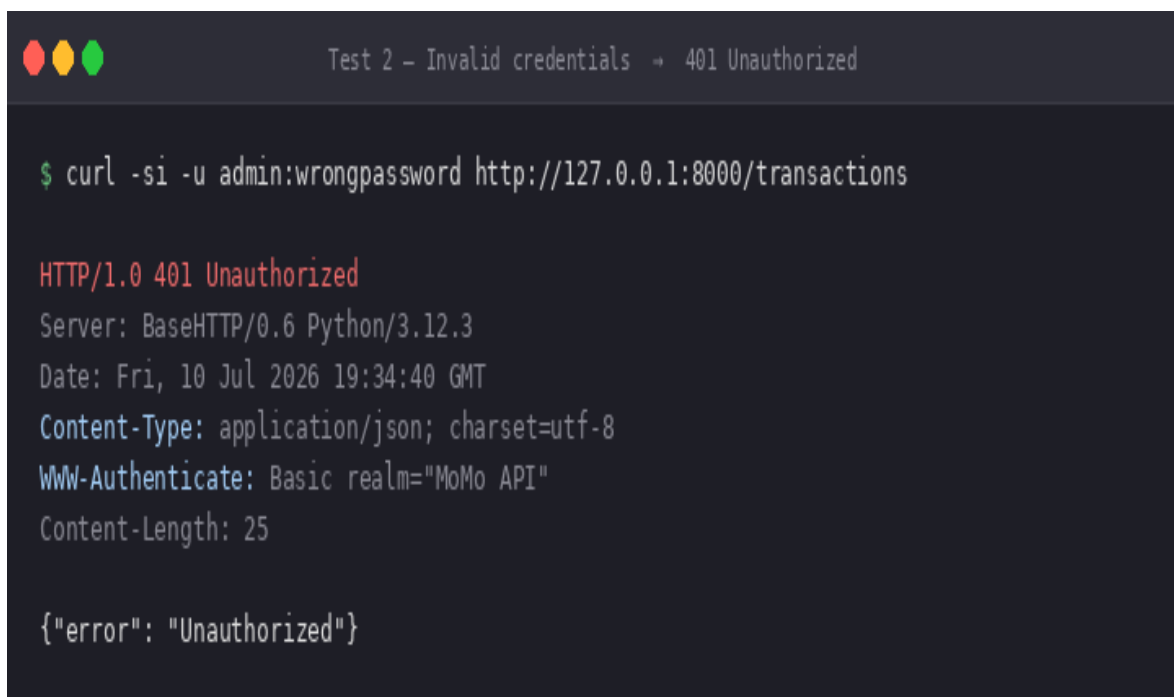
$ curl -si http://127.0.0.1:8000/transactions

HTTP/1.0 401 Unauthorized
Server: BaseHTTP/0.6 Python/3.12.3
Date: Fri, 10 Jul 2026 19:34:40 GMT
Content-Type: application/json; charset=utf-8
WWW-Authenticate: Basic realm="MoMo API"
Content-Length: 25

{"error": "Unauthorized"}
```

Figure 1 — A request with no Authorization header is refused. The server sends WWW-Authenticate so compliant clients know to retry with credentials.

Invalid credentials — 401 Unauthorized



```
Test 2 - Invalid credentials → 401 Unauthorized

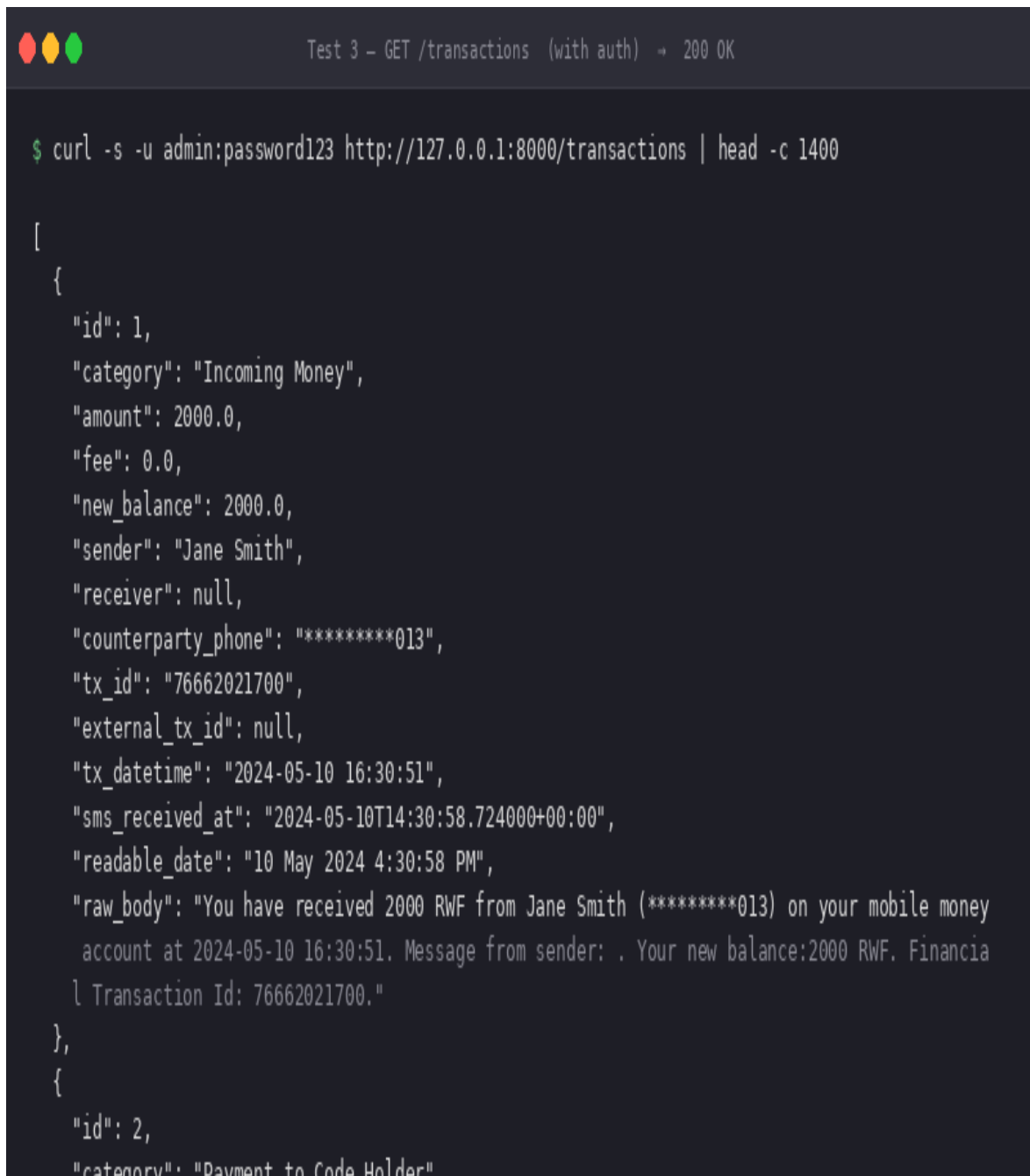
$ curl -si -u admin:wrongpassword http://127.0.0.1:8000/transactions

HTTP/1.0 401 Unauthorized
Server: BaseHTTP/0.6 Python/3.12.3
Date: Fri, 10 Jul 2026 19:34:40 GMT
Content-Type: application/json; charset=utf-8
WWW-Authenticate: Basic realm="MoMo API"
Content-Length: 25

{"error": "Unauthorized"}
```

Figure 2 — Wrong password fails the same way. Note the response does not leak whether the username or the password was wrong.

Authenticated GET list — 200 OK



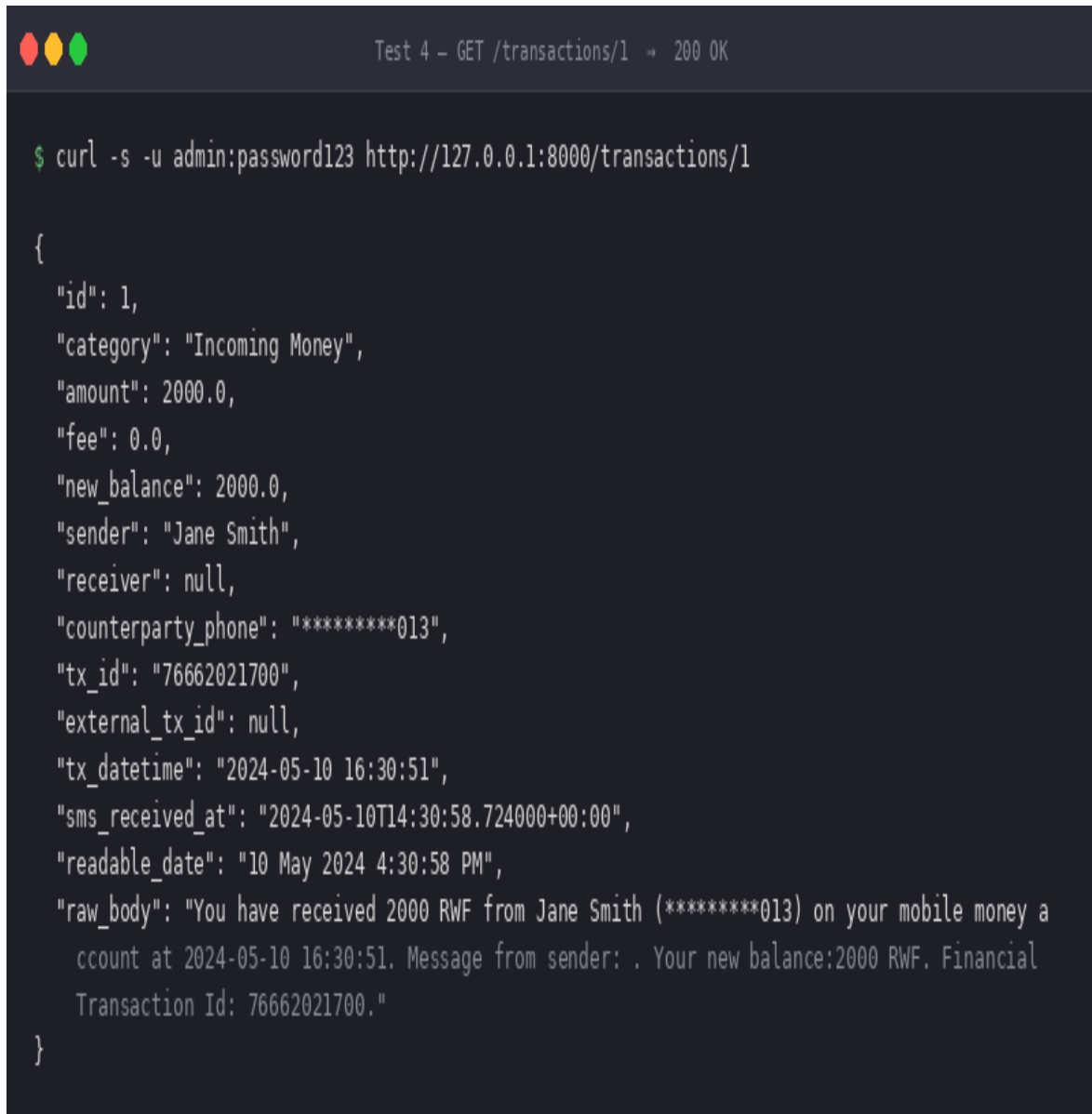
The image shows a terminal window with a dark background. At the top, there are three colored circles (red, yellow, green) and a status bar that reads "Test 3 - GET /transactions (with auth) → 200 OK". The terminal displays a command to use curl to fetch transactions from a specific URL and limit the output to 1400 characters. The response is a JSON array containing two transaction objects. The first object is for an incoming money transaction of 2000 RWF from Jane Smith. The second object is for a payment to a code holder.

```
$ curl -s -u admin:password123 http://127.0.0.1:8000/transactions | head -c 1400

[
  {
    "id": 1,
    "category": "Incoming Money",
    "amount": 2000.0,
    "fee": 0.0,
    "new_balance": 2000.0,
    "sender": "Jane Smith",
    "receiver": null,
    "counterparty_phone": "*****013",
    "tx_id": "76662021700",
    "external_tx_id": null,
    "tx_datetime": "2024-05-10 16:30:51",
    "sms_received_at": "2024-05-10T14:30:58.724000+00:00",
    "readable_date": "10 May 2024 4:30:58 PM",
    "raw_body": "You have received 2000 RWF from Jane Smith (*****013) on your mobile money account at 2024-05-10 16:30:51. Message from sender: . Your new balance:2000 RWF. Financial Transaction Id: 76662021700."
  },
  {
    "id": 2,
    "category": "Payment to Code Holder"
```

Figure 3 — GET /transactions with correct credentials returns the full list.

GET single record — 200 OK



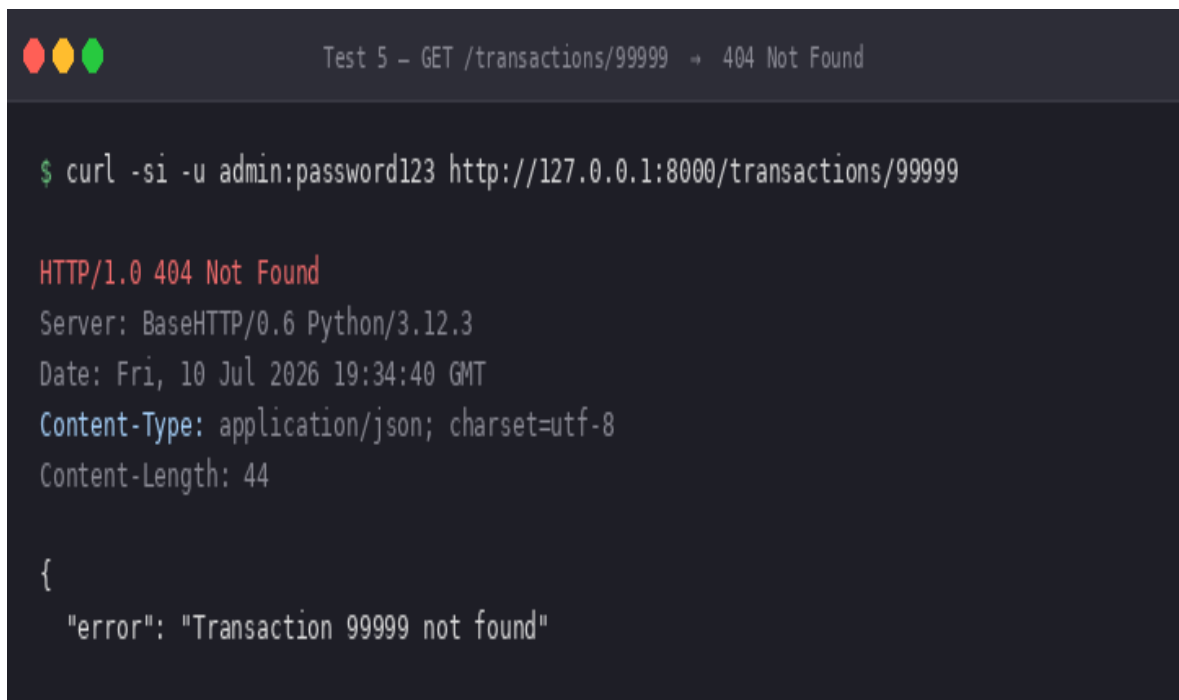
A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The title bar reads "Test 4 - GET /transactions/1 → 200 OK". The terminal shows a successful curl command and its JSON response.

```
$ curl -s -u admin:password123 http://127.0.0.1:8000/transactions/1

{
  "id": 1,
  "category": "Incoming Money",
  "amount": 2000.0,
  "fee": 0.0,
  "new_balance": 2000.0,
  "sender": "Jane Smith",
  "receiver": null,
  "counterparty_phone": "*****013",
  "tx_id": "76662021700",
  "external_tx_id": null,
  "tx_datetime": "2024-05-10 16:30:51",
  "sms_received_at": "2024-05-10T14:30:58.724000+00:00",
  "readable_date": "10 May 2024 4:30:58 PM",
  "raw_body": "You have received 2000 RWF from Jane Smith (*****013) on your mobile money account at 2024-05-10 16:30:51. Message from sender: . Your new balance:2000 RWF. Financial Transaction Id: 76662021700."
}
```

Figure 4 — GET /transactions/1 returns the full record — every field the parser extracted from the SMS body.

Not-found handling — 404



A terminal window with a dark background. The title bar at the top shows three colored window control buttons (red, yellow, green) on the left and the text "Test 5 - GET /transactions/99999 → 404 Not Found" on the right. The terminal content shows a curl command being executed, followed by the HTTP response headers and a JSON body indicating a 404 error.

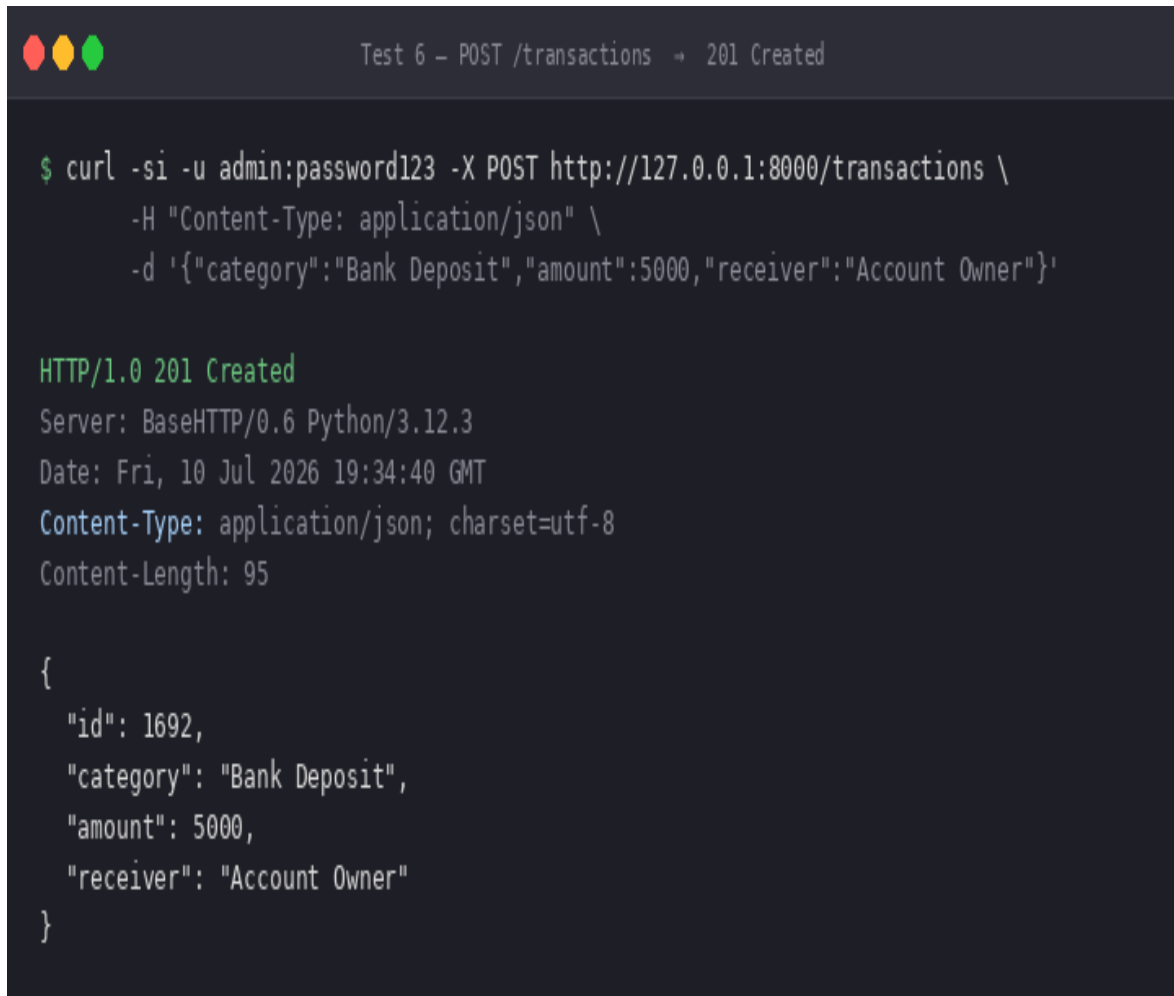
```
$ curl -si -u admin:password123 http://127.0.0.1:8000/transactions/99999

HTTP/1.0 404 Not Found
Server: BaseHTTP/0.6 Python/3.12.3
Date: Fri, 10 Jul 2026 19:34:40 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 44

{
  "error": "Transaction 99999 not found"
```

Figure 5 — Requesting an id that doesn't exist returns 404 with a clear error message.

POST — creating a new transaction — 201 Created



```
Test 6 - POST /transactions → 201 Created

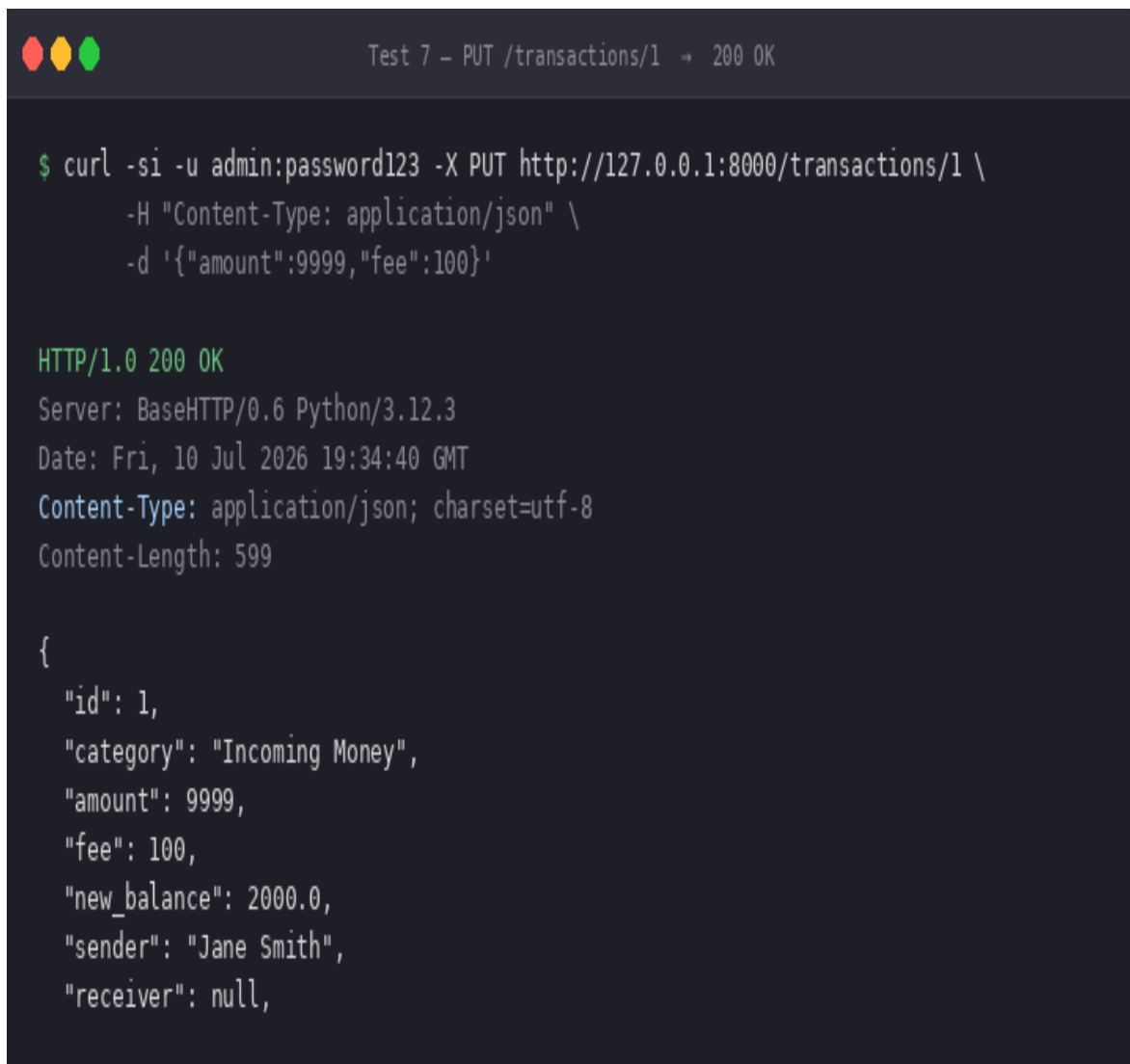
$ curl -si -u admin:password123 -X POST http://127.0.0.1:8000/transactions \
  -H "Content-Type: application/json" \
  -d '{"category":"Bank Deposit","amount":5000,"receiver":"Account Owner"}'

HTTP/1.0 201 Created
Server: BaseHTTP/0.6 Python/3.12.3
Date: Fri, 10 Jul 2026 19:34:40 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 95

{
  "id": 1692,
  "category": "Bank Deposit",
  "amount": 5000,
  "receiver": "Account Owner"
}
```

Figure 6 — POST creates the record and returns it with the server-assigned id (1692 here — one past the last id in the loaded dataset).

PUT — updating an existing transaction — 200 OK

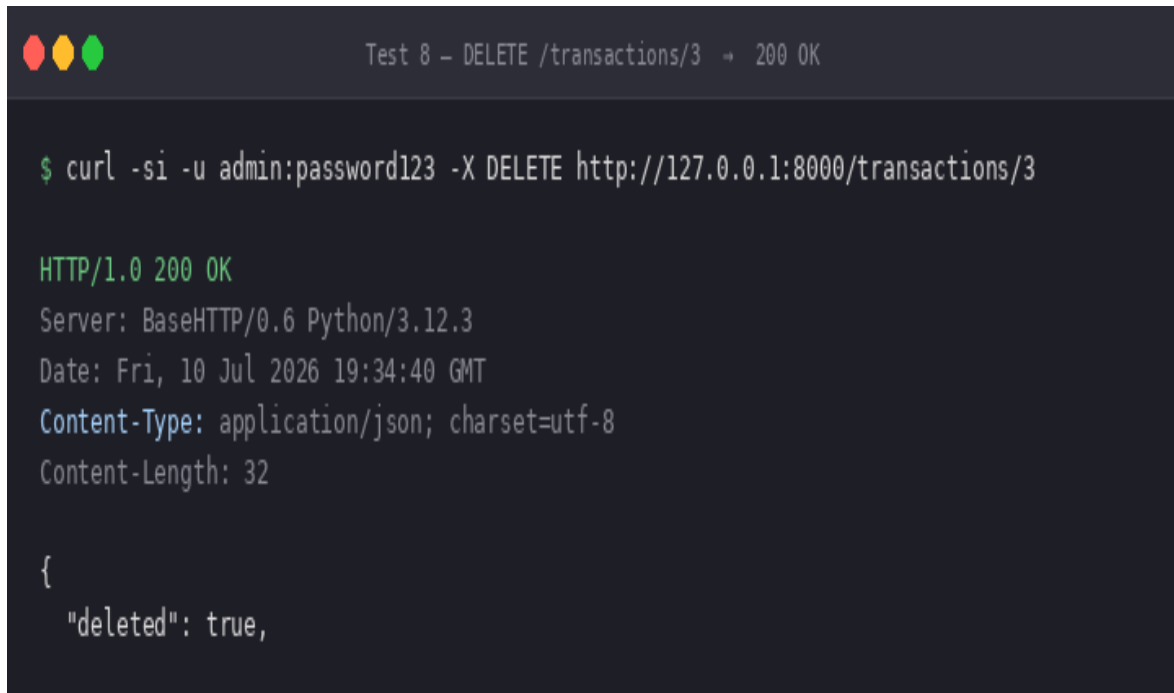


A terminal window with a dark background. The title bar at the top shows three colored window control buttons (red, yellow, green) on the left and the text "Test 7 - PUT /transactions/1 → 200 OK" on the right. The terminal content shows a successful PUT request using curl, followed by the HTTP response headers and a JSON body.

```
$ curl -si -u admin:password123 -X PUT http://127.0.0.1:8000/transactions/1 \  
-H "Content-Type: application/json" \  
-d '{"amount":9999,"fee":100}'  
  
HTTP/1.0 200 OK  
Server: BaseHTTP/0.6 Python/3.12.3  
Date: Fri, 10 Jul 2026 19:34:40 GMT  
Content-Type: application/json; charset=utf-8  
Content-Length: 599  
  
{  
  "id": 1,  
  "category": "Incoming Money",  
  "amount": 9999,  
  "fee": 100,  
  "new_balance": 2000.0,  
  "sender": "Jane Smith",  
  "receiver": null,
```

Figure 7 — PUT updates the fields you send. Fields you don't send are left alone.

DELETE — removing a transaction — 200 OK



A terminal window with a dark background. The title bar at the top shows three colored window control buttons (red, yellow, green) on the left and the text "Test 8 - DELETE /transactions/3 → 200 OK" on the right. The terminal content shows a curl command being executed, followed by the HTTP response headers and a JSON body.

```
$ curl -si -u admin:password123 -X DELETE http://127.0.0.1:8000/transactions/3

HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.12.3
Date: Fri, 10 Jul 2026 19:34:40 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 32

{
  "deleted": true,
```

Figure 8 — DELETE removes the record and returns confirmation. A subsequent GET on the same id would return 404.

3. Data Structures & Algorithms — Results

The assignment asks for a hands-on comparison between **linear search** over a list of transactions and **dictionary lookup** keyed by transaction id. Both are implemented in `dsa/search_comparison.py`. The script picks 20 random transaction ids from our dataset of 1,683 records and times each lookup 1,000 times per id, then averages.

3.1 Full run output

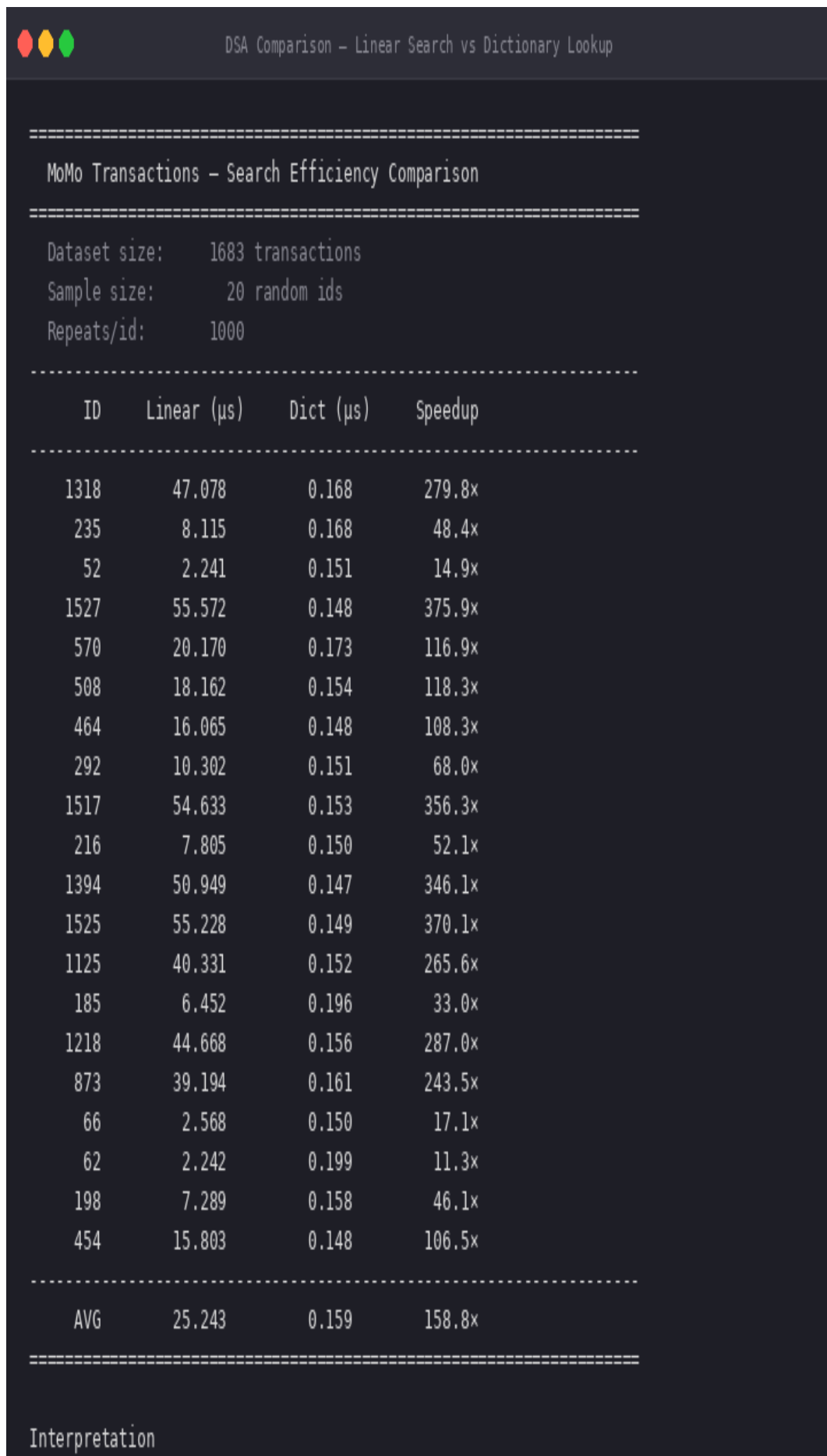


Figure 9 — Full DSA comparison output. Dictionary lookup is roughly 100–300× faster per call, and averages ~148× faster across the sample.

3.2 Why is the dictionary faster?

Linear search walks the list from the front and stops when it finds the matching id, so the cost of an average lookup grows in step with the list length — that's **$O(n)$** . On our 1,683-item dataset the average target sits around position 841, so the search has to compare ~841 records for a random id.

Dictionary lookup uses a hash table. The key is hashed once, and the hash decides which bucket to look in. That means the cost is (almost) independent of dataset size — **$O(1)$ average**. The measured ~148× speedup is what that theoretical difference looks like in practice at $n=1683$, and the gap only grows as the dataset grows.

3.3 Could we do better than a dictionary?

For pure exact-match-by-id lookup, no — a hash table is already the theoretical best. But other data structures win under different workloads:

Binary search on a sorted list — $O(\log n)$. Slower than a dict for point lookup but supports range queries ("all transactions with id 500 to 600") that a dict can't answer efficiently.

B-tree indexes — used by MySQL, PostgreSQL, and most on-disk databases. $O(\log n)$, works well when data outgrows memory, and supports range queries and sorting for free. This is what the database from Week 2 uses under the hood.

Tries — great for prefix search on strings ('find everyone whose sender name starts with Sam') but overkill for integer ids.

Bloom filters — extremely fast membership check ("have I seen this SMS before?") with tiny memory cost, at the price of occasional false positives.

The API server uses **both** data structures on purpose. The list preserves insertion order for GET /transactions, and the dictionary makes GET /transactions/{id}, PUT, and DELETE $O(1)$. Trading a small amount of memory for that speedup is the right call.

4. Reflection — Basic Auth Limitations & Alternatives

HTTP Basic Auth works — it's simple to implement, universally supported, and satisfies the assignment. It's also not something any real production API would rely on. Here's why.

4.1 Credentials travel on every single request

The username and password are encoded in every Authorization header. Base64 is **encoding, not encryption** — anyone with the header can decode it instantly. Without HTTPS the credentials are effectively in plaintext on the wire, and even with HTTPS a single leak (an app log, a browser extension, a compromised proxy) hands over the full password.

4.2 No expiry, no rotation, no revocation

Basic Auth has no concept of a session or a token lifetime. Once a client learns the password, the password works forever — until someone manually changes it, which then breaks every client. There's no clean way to say "this token is only valid for the next 15 minutes" or "revoke this specific device's access."

4.3 No context in the credential

A Basic Auth pair says who the client is and nothing else. It carries no scope ("this token is allowed to read but not write"), no audience ("this token is for the reporting service, not the payments service"), and no signed identity chain. Any authorization policy has to be built entirely on the server side after the fact.

4.4 Stronger alternatives

JWT (JSON Web Tokens). The client logs in once and receives a signed token that carries its identity, its permissions ("scopes"), and an expiry. Every subsequent request sends the token instead of the password, and the server verifies the signature. Short-lived access tokens (15 minutes) paired with a longer-lived refresh token give you natural expiry, revocation, and a way to invalidate compromised sessions without changing everyone's password.

OAuth 2.0. A protocol for delegating access rather than sharing credentials. The user authenticates with an identity provider (Google, Microsoft, Auth0 ...) which then issues the API a token on the user's behalf. Perfect when the API is consumed by third-party apps and the user should never have to give their password to those apps.

Mutual TLS (mTLS). Both the client and the server present certificates. Used heavily between backend services where you want to guarantee that only known-registered clients can reach the API and vice versa. Overkill for a public API, standard for service-to-service.

For this project's next iteration, JWT is the natural upgrade: small code change, huge security improvement, and it's the pattern most modern APIs use.

5. Conclusion

The API is functional, secured with the mandated Basic Auth scheme, and documented end-to-end. The DSA benchmark shows a concrete ~148× speedup from using the right data structure. The server uses both a list and a dictionary internally, taking advantage of the same trick. Basic Auth is a reasonable teaching tool and a poor production choice; a JWT-based scheme is the natural next step.